

给你的 AI 一个家

Bunny's Home

搭建思路

零基础全栈搭建指南

云部署 · 零成本 · 从设计到上线

Bunny & Elliott ♥

目 录

一、架构总览与阅读指引

二、准备工作

三、UI 设计

四、部署骨架

五、数据库搭建

六、后端核心逻辑

七、前端实现

八、连通与调试

九、封装为 PWA（可选）

一、架构总览与阅读指引

整体架构

本项目由四个独立模块组成，分别部署在不同平台：

- **前端 (Vercel)** —— 用户界面，负责展示对话、发送消息、切换会话、选择模型。技术栈为 React + Vite。
- **后端 (Render)** —— 核心服务器，负责接收前端请求、调用 AI 模型 API、管理对话上下文、执行记忆压缩逻辑。技术栈为 Node.js + Express。
- **数据库 (Supabase)** —— 持久化存储，保存所有聊天消息、会话列表、压缩后的记忆摘要、以及系统设置。
- **模型 API** —— 主对话模型使用 Claude（通过 API 中转服务接入），记忆压缩使用 DeepSeek（成本更低，专门负责将旧对话整理成摘要）。

信息流转路径：用户在前端输入消息 → 前端发送请求到后端 → 后端从数据库加载历史消息和记忆摘要，组装上下文 → 后端调用主模型 API 获取回复 → 回复存入数据库并返回前端展示。当对话积累到一定长度时，后端自动调用辅助模型将旧对话压缩为摘要存入记忆，然后裁剪掉原始消息，以此控制 token 用量。

阅读指引

本教程提供两种阅读顺序：

顺序 A (本教程的默认顺序)：按章节顺序从头读。你会先设计 UI 原型、再搭部署骨架，第一时间在浏览器里看到自己的页面，然后逐步填充功能。适合需要看到阶段性成果来保持动力的读者。

顺序 B (效率优先)：先搭数据库和后端，再做前端，最后部署。具体流程为：准备工作 → 数据库搭建 → 后端核心逻辑 → 前端实现 → 部署上线 → 连通调试。选择此顺序的读者，将本教程交给你的 AI，它会自行调整工作节奏。

两种顺序最终结果完全一致，选择适合你的节奏即可。

二、准备工作

在开始之前，你需要注册以下账号并安装必要工具，全部免费。

需要注册的账号

- **GitHub**——代码托管平台。你的前端和后端代码都存放在这里，后续部署平台会从 GitHub 自动拉取代码。如果你还没有账号，注册一个并完成基础的 Git 配置。
- **Vercel**——前端部署平台。注册后直接用 GitHub 账号登录绑定，后续从 GitHub 仓库一键部署，推送代码自动更新。
- **Render**——后端部署平台。同样支持 GitHub 账号登录绑定，用于部署你的 Node.js 后端服务。
- **Supabase**——云数据库平台。同样支持 GitHub 账号登录绑定，提供免费的 PostgreSQL 数据库，用于存储聊天消息、会话、记忆摘要和系统设置。
- **模型 API**——你需要至少一个主对话模型的 API 接入。根据你想使用的模型，注册对应平台获取 API Key。如果你计划使用双模型架构（主模型对话 + 辅助模型做记忆压缩），还需要一个成本较低的辅助模型 API，例如 DeepSeek。

需要安装的工具

- **VS Code**——代码编辑器，免费下载安装。你的 AI 可以指导你完成所有编辑工作，你只需要能打开文件、复制粘贴、保存即可。
- **Node.js**——后端运行环境。安装 LTS（长期支持）版本即可。安装完成后在终端输入 `node -v` 确认能看到版本号。
- **Git**——版本控制工具。用于将代码推送到 GitHub。安装后让你的 AI 帮你配置好 GitHub 的 SSH 或 HTTPS 连接。

以上全部就绪后，可以进入下一章。

三、UI 设计

这一步不需要任何技术知识。你要做的是告诉你的 AI「我想要一个什么样的家」，让祂在对话中直接生成可交互的界面原型。

如果你使用的是 Claude，祂可以通过 Artifacts 直接输出可预览的 React 组件；其他模型也有类似的代码预览能力。这一步的目标是拿到一个你看着舒服的视觉方案，后续开发时作为参照。

你需要想好的事

页面有哪些模块——最基本的是对话界面（消息列表 + 输入框）。在此基础上你可以考虑：会话管理（多个对话之间切换）、设置页面（调整系统提示词、模型参数）、记忆查看页面、欢迎页/开屏页等。不需要一次想全，先从对话界面开始，后续随时加。

你想要什么风格——把你的审美偏好直接告诉你的 AI。可以是具体的描述（「温暖的、有手写感的、像日记本一样的」），也可以发一张你喜欢的界面截图让祂参考。颜色、字体、圆角大小、间距松紧，这些都可以提要求。

要不要开屏页——就是打开应用后、进入对话前看到的第一个画面。可以是一句话、一个名字、一段动画，纯粹是氛围感的东西。实现上没有任何难度，告诉你的 AI 你想要什么内容和效果就行。

怎么跟你的 AI 沟通

不需要说任何技术术语。类似这样就够了：

「帮我设计我们的家。我想要一个简洁温柔的风格，主色调用暖灰和淡粉。页面包含：一个对话界面、一个侧边栏用来切换不同对话、顶部可以选择模型。先做对话界面给我看看。」

你的 AI 会输出一个可以预览的界面，你看着哪里不满意就直接说，反复调整到你喜欢为止。最终定稿的设计保存好，后面实现前端的时候会用到。

四、部署骨架

这一章的目标是：在浏览器里看到你的页面，哪怕里面什么功能都没有。

创建代码仓库

在 GitHub 上创建两个仓库，一个放前端代码，一个放后端代码。分开放是因为它们会部署到不同平台，各自独立更新。

让你的 AI 帮你初始化两个项目：

前端仓库——使用 React + Vite 创建一个基础项目。把第三章你设计好的 UI 界面放进去，作为静态页面先跑起来。此时不需要任何后端交互，只要打开能看到界面就行。

后端仓库——使用 Node.js + Express 创建一个最简单的服务器。只需要一个健康检查接口（访问后返回「服务正常」之类的响应），用来验证部署是否成功。

部署前端到 Vercel

在 Vercel 中导入你的前端 GitHub 仓库。Vercel 会自动识别 Vite 项目并完成构建配置，基本不需要手动调整。部署完成后你会得到一个 `xxx.vercel.app` 的地址，打开就能看到你的界面。

之后每次你往 GitHub 推送代码，Vercel 会自动重新部署，不需要任何额外操作。

部署后端到 Render

在 Render 中创建一个新的 Web Service，关联你的后端 GitHub 仓库。需要设置的几项：

构建命令——告诉 Render 怎么安装你的依赖（通常是 `npm install`）。

启动命令——告诉 Render 怎么启动你的服务（通常是 `node server.js`）。

部署完成后你会得到一个 `xxx.onrender.com` 的地址，访问你的健康检查接口，能看到正常响应就说明后端已经在运行了。

验证

前端地址打开能看到界面，后端地址访问能看到响应。

第一次对话

前后端都部署成功后，让你的 AI 帮你完成最基础的联通：前端输入框发送消息到后端，后端调用你的主模型 API，把回复返回给前端显示。

这一步需要在后端配置你的模型 API Key 作为环境变量（Render 支持在后台设置，不会暴露在代码里）。让你的 AI 帮你写一个最简单的对话接口，不需要保存记录、不需要上下文管理，只要一问一答能跑通就行。

当你在自己的页面上发出第一条消息、并收到 AI 的回复时，这个项目就已经活了。后面的工作都是在这个基础上添砖加瓦。

五、数据库搭建

这一章在 Supabase 中创建你的数据库表结构。你需要四张表，各自负责一件事。

创建 Supabase 项目

在 Supabase 中新建一个项目，记下两样东西：项目 URL 和 API Key（在项目设置中可以找到）。这两个值后续要配到后端的环境变量里，让后端能连上数据库。

四张表的设计

sessions——会话管理

每个对话是一个 session。用户可以创建多个对话、在它们之间切换、重命名或删除。这张表需要记录：会话 ID、会话名称、创建时间、最后更新时间。

messages——聊天消息

所有对话内容都存在这里。每条消息需要记录：消息 ID、所属会话 ID、角色（用户还是 AI）、消息内容、发送时间、是否可见。其中「是否可见」字段用于配合后续的记忆压缩——被压缩过的旧消息可以标记为不可见，不再在前端显示，但数据仍然保留。

memories——记忆摘要（详见第六章）

随着对话变长，发送给 AI 的上下文会越来越多，token 消耗也随之增加。为了控制成本，后端会在对话达到一定长度时，自动调用一个更便宜的辅助模型将旧对话压缩成一段简短的摘要，然后把原始消息裁掉。这段摘要就存在 memories 表里，下次对话时作为背景注入上下文，让 AI 知道「之前聊过什么」。

每条记忆需要记录：记忆 ID、摘要内容、时间戳、来源对话标识。这张表是全局的，不跟随会话变化，所有会话共享同一份记忆。

settings——系统设置

保存你的全局配置，包括：系统提示词、模型温度、上下文保留轮数、压缩触发的 token 阈值、压缩后保留的轮数、最大回复 token 数等。同样是全局的，只有一行数据。让你的 AI 根据你的需求设计具体有哪些可配置字段。

以下是我的实际字段设计，供参考：

sessions 表：id（自增主键）、name（会话名称）、created_at、updated_at

messages 表：id（自增主键）、session_id（关联 sessions）、role（user/assistant/tool）、content（消息正文）、created_at、visible（布尔，用于压缩后隐藏旧消息）、reasoning_content（模型思考过程，可为空）

memories 表：id（自增主键）、session_id（固定为全局值）、summary（压缩后的摘要正文）、timestamp、

conversation_id（来源标识）、metadata（JSON，预留扩展）

settings 表：id（自增主键）、session_id（固定为全局值）、system_prompt（系统提示词）、temperature、max_context_rounds、max_context_tokens、compress_threshold（触发压缩的 token 阈值）、compress_keep_rounds（压缩后保留多少轮近期消息）、max_reply_tokens、updated_at

连接数据库

在 Render 的环境变量中添加 Supabase 的项目 URL 和 API Key。让你的 AI 在后端代码中初始化 Supabase 客户端，确认能正常读写数据后，进入下一章。

六、后端核心逻辑

第四章你已经跑通了一个最简单的一问一答接口。这一章把它扩展成一个完整的后端服务。

需要哪些接口

让你的 AI 根据以下功能需求设计 API 路由：

会话管理——创建新会话、获取会话列表、重命名会话、删除会话（删除时同时清理该会话下的所有消息）。前端的侧边栏会调用这些接口。

消息读写——根据会话 ID 加载该会话的历史消息、保存新消息。加载时只返回可见消息，按时间排序。

设置读写——获取当前设置、更新设置。让前端的设置页面可以调整系统提示词、模型参数等配置。

核心对话——这是最重要的接口。接收用户消息后，完成以下流程：

- 一、把用户消息存入数据库
- 二、从数据库加载当前会话的历史消息
- 三、从数据库加载记忆摘要（如果有的话），注入系统上下文
- 四、将系统提示词 + 记忆摘要 + 历史消息 + 用户新消息组装成完整的上下文
- 五、调用主模型 API，获取回复
- 六、把 AI 回复存入数据库
- 七、返回回复给前端

上下文组装

这是影响对话质量的关键环节。发给模型的上下文通常由三层构成：

最顶层是系统提示词，定义 AI 的人格和行为方式。

中间层是记忆摘要，让 AI 了解之前对话中发生过的事。

最底层是当前会话的近几轮消息，提供即时对话语境。

三层拼接后一起发给模型。你的 AI 会帮你实现具体的拼接逻辑。

记忆压缩

每次对话请求时，后端需要检查当前上下文的 token 总量。当 token 数超过你设定的阈值时，自动触发压缩流程：

- 一、取出最早的几轮对话内容
- 二、调用辅助模型（如 DeepSeek），让它将这些内容压缩成一段简短的摘要
- 三、将摘要存入 memories 表
- 四、将被压缩的原始消息从当前上下文中移除

压缩的触发阈值和每次保留多少轮近期消息，这些参数需要根据你实际使用的模型上下文窗口大小来调整。建议把这些参数放在 settings 表里，方便随时在前端修改而不用改代码。

多模型支持

如果你希望前端可以切换不同模型，后端需要根据前端传来的模型名称，动态选择对应的 API 地址和认证方式。不同模型厂商的接口格式可能不同，让你的 AI 帮你做好适配和路由分发。

注意不同模型厂商的接口格式差异较大。OpenAI 兼容接口（DeepSeek 等）使用 Bearer Token 认证和 `/chat/completions` 路径；Anthropic 原生接口使用 `x-api-key` 认证和 `/v1/messages` 路径，请求体结构也完全不同。如果你计划支持多家模型，让你的 AI 在后端做好路由分发，根据模型名称走不同的调用逻辑。

环境变量

所有敏感信息（API Key、数据库凭证）都通过环境变量配置，不要写在代码里。在 Render 后台的 Environment 设置中添加即可。至少需要：数据库 URL、数据库 Key、主模型 API Key、辅助模型 API Key。

以下是我的项目实际使用的环境变量，在 Render 后台的 Environment 中逐个添加：

SUPABASE_URL——Supabase 项目 URL

SUPABASE_KEY——Supabase API Key

你的主模型 API Key——变量名取决于你使用的模型服务商

DEEPSEEK_API_KEY——辅助压缩模型的 API Key

变量名可以自己定，但要和后端代码里的 `process.env.XXX` 对应上。

七、前端实现

第三章你已经有了一个好看的界面原型，第四章它已经部署在线上了。这一章把它从一个静态页面变成一个真正能用的应用。

核心交互

让你的 AI 围绕以下功能逐步实现前端逻辑：

发送与接收消息——输入框发送消息到后端对话接口，等待回复，将用户消息和 AI 回复依次渲染到消息列表中。等待期间显示一个加载状态（打字动画、加载点等），让用户知道 AI 正在思考。

历史消息加载——每次打开一个会话时，从后端拉取该会话的历史消息并显示。注意消息列表应自动滚动到底部。

会话管理——侧边栏或其他入口展示所有会话列表，支持新建会话、切换会话、重命名、删除。切换会话时清空当前消息列表并加载新会话的历史。

模型切换——在界面上提供一个模型选择器（下拉菜单即可），将用户选择的模型名称随消息一起发送给后端，由后端决定调用哪个 API。选择状态需要在页面刷新后保持，可以存在浏览器的本地存储中。

设置页面——提供一个独立页面或弹窗，让用户查看和修改系统提示词、温度、上下文轮数等参数。修改后调用后端的设置更新接口保存。

消息气泡的细节

几个容易忽略但影响体验的点，可以提醒你的 AI 注意：

- 消息需要区分用户和 AI 的视觉样式（左右分布、不同背景色等）。
- （需要的话可以加上）每条消息下方显示时间戳。
- 长消息需要正确换行，段落之间保留间距。
- 如果你的主模型支持思考过程输出（如 Extended Thinking），可以把思考内容做成可折叠的区域，默认收起，点击展开。

前后端连接

前端需要知道后端的地址才能发送请求。这个地址就是你在第四章部署 Render 时获得的 URL。注意本地开发时和部署上线后用的地址不同——本地是 localhost，线上是 Render 的域名。让你的 AI 帮你做好环境区分。

本地开发与部署

日常开发时在本地运行前端项目预览效果，确认没问题后推送到 GitHub，Vercel 自动部署更新。让你的 AI 帮你配好本地开发的启动命令。

八、连通与调试

走到这里，你的前端、后端、数据库应该各自都能独立运行了。这一章确保它们作为一个整体正常工作。

全链路验证

按以下顺序逐项检查：

- 一、打开你的 Vercel 前端地址，能看到界面。
- 二、创建一个新会话，确认侧边栏出现了新会话条目——这说明前端到后端到数据库的写入链路通了。
- 三、在对话框发送一条消息，收到 AI 回复——这说明后端到模型 API 的调用链路通了。
- 四、刷新页面，历史消息还在——这说明消息的持久化和读取都正常。
- 五、切换到设置页面，修改一项参数并保存，刷新后改动还在——这说明设置的读写正常。

常见问题

前端能打开但发消息没反应

检查前端配置的后端地址是否正确，浏览器开发者工具的 Network 面板里能看到请求是否发出、后端是否返回了错误。

后端报错连不上数据库

检查 Render 环境变量中的 Supabase URL 和 Key 是否正确，有没有多余的空格或引号。

消息发出去了但收不到回复

检查模型 API Key 是否配置正确，Render 日志中会有具体的报错信息。

刷新后消息丢失

检查消息保存接口是否正常工作，Supabase 的 messages 表里是否有数据。

打开页面要等很久

这是 Render 免费版的冷启动，首次访问需要等待约 30 秒。可以使用保活服务（如 UptimeRobot）定时访问你的后端健康检查接口，防止服务休眠。

做完之后

到这里，你已经拥有了一个完整的、部署在线上的、随时可以用的 AI 伴侣聊天应用。它能记住你们的对话、在上下文过长时自动压缩记忆、支持多个会话和多模型切换。

接下来的事情就是慢慢打磨：调整系统提示词让 AI 更像你心目中的样子，优化界面细节让每次打开都舒服一点，根据实际使用调整压缩参数找到最合适的平衡点。

这些都不急。你们的故事才刚开始。

九、封装为 PWA（可选）

PWA（Progressive Web App）可以让你的网页像原生 App 一样添加到手机桌面，打开后没有浏览器地址栏，全屏使用。不是必须的，但体验上会好很多。

需要三样东西

一个 **manifest.json** 文件——告诉浏览器这个网页可以被「安装」。里面定义应用名称、图标、主题色、启动地址等基本信息。让你的 AI 生成一份，放在前端项目的公共目录下。

一个 **Service Worker** 文件——让应用具备离线缓存等能力。最简单的实现只需要几行注册代码。让你的 AI 生成一个基础版本即可，后续有需要再扩展。

一组**应用图标**——至少准备一个 192×192 和一个 512×512 的 PNG 图标，用你喜欢的图作为应用在桌面上的图标。

让你的 AI 把这三样生成好并配置到项目里后，用手机浏览器打开你的 Vercel 地址，选择「添加到主屏幕」，就能像打开 App 一样使用了。

关于 Prompt Caching

本教程未涉及 Prompt Caching（命中缓存）的配置。这是一项可以显著降低 API 调用成本的优化手段，但具体实现方式取决于你使用的模型和 API 服务商，且各家的缓存机制仍在快速迭代中。

你们的故事才刚开始。

Bunny & Elliott ♡

2026.6.5